

QK2 Card Protocol

The card is a **bearer object**. Two interchangeable roles — **B (Key Card)** and **C (Emergency Card)** — run the **same minimal applet**: an independent, terminal-provisioned **BIP48 account key (xprv at m/48'/0'/0'/2')** that never leaves, the shared exportable **A2** key, and the wallet descriptor/policy (D). Physical custody is authorization; the 2-of-3 wallet policy is the spending control. There is **no PIN, no OAC, no duress, no lockout, no bricking, and no terminal pairing**. The terminal is interchangeable and holds no persistent wallet secret. Every card-side primitive remains gated on a real capability matrix that does not yet exist.

PROJECT STATUS — THIS DRAFT DOES NOT CLOSE GATE B

Every primitive here is **SELECTED product direction, not implemented**, and gated on a real card capability matrix and bench proof (§02). Do not infer secp256k1 ECDH, HMAC, AEAD, or transactional behaviour from generic JavaCard API names — API constants can exist while the physical card returns NO_SUCH_ALGORITHM . In v1 the card bus sits **inside the trusted terminal boundary**, so no secret channel is added: T=1 needs integrity and error handling, not fleet keys or pairing. A standard card-authenticated profile (an appropriate **SCP11** profile with no terminal pairing or shared host secret) is possible **LATER hardening only**, never a recovery dependency and never user authentication. Card capability, endurance, reset, tear, APDU size, and timing are Gate-B bench blockers. The retired custom-channel construction and the deferred anti-exfil material are confined to clearly non-normative appendices (A–B). A Python example proves math, not silicon.

BRIDGE	LINK	ROLES	CHANNEL	ACCOUNT XPRV	A2	PIN/OAC/LOCKOUT
SEC1210 · UART	ISO 7816-3 T=1	B & C · same applet	trusted bus (v1) · SCP later	never export	exportable by design	none

PAIRING
none –
terminal
interchangeable

1. [Topology & threat model](#)
2. [Card capability matrix \(bench-required\)](#)
3. [The session: roles, session, operations](#)
4. [Provisioning ceremony](#)
5. [Minimal command set](#)
6. [Threat → posture](#)
7. [Appendix A — custom channel \(retired\)](#)
8. [Appendix B — anti-exfil leg-B \(deferred\)](#)

01 TOPOLOGY

Four physical surfaces between the app CPU and the applet

The terminal MCU talks to a Microchip SEC1210 over UART; the SEC1210 drives the ISO-7816 card (VCC/CLK/RST/IO). Those are two physical buses — the UART and the card contacts — but in v1 both sit **inside the trusted terminal boundary**. The residual threats the protocol actually reasons about are a **hostile terminal** and a **cloned card**. The protocol's first principle follows directly.

```
terminal MCU -UART→ SEC1210 -ISO7816→ smart card (JavaCard applet)
    |               |               |
    └─ tap #1 (UART) ─┬─ the bridge ─┬─ tap #2 (card contacts)
                    (must be transparent) clone / counterfeit card
```

In v1 the card bus is treated as part of the **trusted terminal physical boundary**; the protocol adds **no bespoke secret channel**. This is deliberate: a bus channel would not fix the dominant threat, because a malicious terminal can already read a released A2 and ask the bearer card to sign. T=1 framing needs integrity and error handling, not secret fleet keys. If a selected card supports a standard, publicly specified, card-authenticated ephemeral channel (an appropriate **SCP11** profile with no terminal pairing or shared host secret), it may be evaluated as **later hardening only** — it must never become a recovery dependency and is **not** user authentication. Terminal possession is **not** a spending factor: the 2-of-3 threshold is.

What the card holds (same minimal applet, roles B and C)

- **Signer B / Signer C** — an **independent BIP48 account private key (xprv at m/48'/0'/0'/2')**, plus chain code and origin fingerprint) that the **terminal derives** from independent 256-bit entropy and provisions onto the card (Signer B on a B card, Signer C on a C card; never derived from Seed A or one master). The applet implements **no BIP39 and no PBKDF2**: it derives only the non-hardened receive/change children and signs on-card. **Never exportable, by design.**
- **A2** — the fresh 256-bit random key that decrypts the Recovery Document (A1). The **same A2 is duplicated onto both B and C** at provisioning. A2 is **EXPORTABLE by design** (it is a bearer recovery object, not a Bitcoin signing key); the replaceable terminal performs the actual AEAD decryption of A1 (ChaCha20-Poly1305, RFC 8439).
- **Wallet descriptor / policy (D)** — key fingerprint/origin, policy ID, and the two checksummed descriptors (receive + change) so `GET_POLICY` can return them. **D is stored on both B and C** — every valid spending pair contains a card, so every valid pair can recover the wallet map. No Vault Key, no capsule authentication on-card.

NOT HELD, NOT ENFORCED

The card holds **no PIN, no OAC/activation verifier, no duress secret, no retry/lockout counter**, and **no terminal pairing key**. An optional attestation identity is a **manufacturing-security** question (a real trust chain with rotation/revocation), not user authentication, and is contingent. Any monotonic counter is telemetry only: **a counter is not clone-detection without an external anchor and an explicit user action.**

02 PRIMITIVES

Card capability matrix – must be measured, not inferred

The single most important correction to this document: **a JavaCard API constant does not prove the physical card implements the algorithm, exposes it in the form needed, or has the RAM/timing/endurance to run it.** `ALG_EC_SVDP_DH` does not guarantee `secp256k1`; `ALG_ECDSA_SHA_256` does not guarantee raw 32-byte-digest signing, caller-controlled nonces, nonce-point output, or low-S; Oracle's AEAD APIs are AES-GCM and AES-CCM, **not AES-GCM-SIV**. The table below is the matrix to *fill with bench results* for the actual card — every row starts **BENCH-REQUIRED**.

CAPABILITY TO PROVE ON THE REAL CARD	PROPOSED PRIMITIVE	STATUS
secp256k1: load provisioned account xprv + non-hardened BIP32 child derivation + persistent storage (card RNG = Multi-source entropy sample only, not signer keygen)	EC secp256k1	BENCH-REQUIRED
raw 32-byte-digest ECDSA · exact deterministic RFC6979 · strict DER · low-S (caller nonce control only for the deferred anti-exfil, App. B)	ECDSA	BENCH-REQUIRED
ECDH output format + point validation (<i>only for optional later SCP11 hardening — not a v1 requirement</i>)	EC-DH	BENCH-REQUIRED
AEAD: which of GCM / CCM (NOT GCM-SIV) + key sizes (<i>only for optional later SCP11 hardening; A1's own AEAD is ChaCha20-Poly1305 on the terminal — not v1</i>)	AES-GCM or AES-CCM	BENCH-REQUIRED
HMAC-SHA-256 and/or AES-CMAC (<i>only for optional later SCP11 hardening — not a v1 requirement</i>)	MAC	BENCH-REQUIRED
transient RAM, APDU limits, per-op timing	—	BENCH-REQUIRED
transaction buffer + atomic-write + monotonic-counter endurance	—	BENCH-REQUIRED

DESIGN DECISION D1 · CHANNEL CRYPTO IS LATER-HARDENING, NOT V1

v1 adds **no bus channel** (the card bus is trusted). MAC/AEAD on the card would matter only if a standard **SCP11** profile is ever added as later hardening; if so, prefer that profile's own **HMAC-SHA-256 or AES-CMAC** integrity — KMAC/SHA-3 is unlikely on the card — confirmed on the card, not from an API name. Separately, the AEAD that decrypts A1 is **ChaCha20-Poly1305 (RFC 8439)** with an A2-derived key and a stored fresh 96-bit nonce, performed by the **replaceable terminal**, not the card: **the permanent A1 document format is not coupled to any AEAD a JavaCard happens to expose**. The card's job is to release A2 over a minimal session; the terminal decrypts.

03 THE SESSION

Roles, session, operations – no gate in between

There is **no authorization phase**. A card in hand is authorized to do its part; the 2-of-3 wallet policy — not any card-side PIN, OAC, or lockout — is the spending control. A session is just: identify the card, then perform an operation. v1 adds no bus channel; a standard card-authenticated channel is later hardening only.

P1 Identify – which role, which key, which policy

GET_INFO returns the **role (B or C)**, protocol/applet version, lifecycle state, card id, wallet id, and the account fingerprint/origin. Optional attestation is a **manufacturing-security** check with a real trust chain + rotation/revocation, **not** user authentication, and is treated as contingent. Any counter the card reports is telemetry: **a counter jump is not clone-detection without an external anchor and a deliberate user action**. No card fails "closed" here — there is nothing to lock out.

P2 **Bus is trusted in v1 – a channel is later hardening only**

v1 treats the card bus as inside the **trusted terminal boundary** and adds no secret channel. If a selected card supports a **standard SCP11 profile** (card-authenticated, no terminal pairing, no shared host secret), it may be evaluated as **later hardening** to encrypt+MAC the wire (Appendix A holds a retired custom construction, not a candidate). Even then it protects the wire only: it is **not** a user-auth claim, must **not** bind the card to one terminal, and must **never be a recovery dependency**. The terminal is interchangeable.

P3 **Operate – release A2, sign, read policy**

Over the session: GET_A2 releases the shared A2 key so the terminal can decrypt A1 — **A2 is exportable by design**; SIGN binds a wallet id, branch {0,1}, bounded child index, sighash, review hash, and session nonce, then signs with the **non-exportable** account key (Signer B or Signer C) and verifies before release; GET_POLICY returns the receive/change descriptor D. There is **no RELEASE-SEED and no on-card Vault Key** — the signing key never leaves the card, and A1 decryption happens on the replaceable terminal.

WHY NO GATE, AND WHY IT MATTERS MOST FOR C

B and C are **bearer objects**: possession is authorization, and the threshold is the real control. This deliberately removes every forgettable dependency — no PIN, no vendor, no account, no clock. It matters most for **C**: an inheritance kit must still work after years, when a PIN would be lost, a vendor gone, and a retry-counter or lockout would be a liability rather than a protection. Losing **two** signing factors can be unrecoverable — this is stated plainly elsewhere in the dossier, and the card protocol adds no recovery of its own.

04 PROVISIONING

One-time write of role, independent account xprv, A2, and D

Provisioning is a one-time write of role, independent account xprv, A2, and D. The card protocol adds no recovery of its own.

Provisioning writes a card's **role (B or C)**, loads the **independent BIP48 account xprv** that the terminal derived from independent 256-bit entropy, receives the shared **A2** that the terminal duplicates onto **both B and C**, stores the wallet descriptor D, and verifies. There is **no ARM step and no authorization verifier** — the card holds no OAC, PIN, or lockout state. The terminal conditions all key material (the card RNG may contribute only a Multi-source entropy sample), writes the same A2 to B and to C, verifies both, then **wipes A2 and all seed/root material from itself**: no preinstalled secret survives on the interchangeable terminal. With primary AAA cells and no RTC, a power cut mid-ceremony must never leave a half-made or doubly-provisioned card; the card runs it as a transactional state machine.

Describing an FSM does not establish power-loss safety — atomic write, resume/rollback, and endurance are Gate-B bench items on the selected card.

NON-NORMATIVE CANDIDATE · DEPENDS ON UNPROVEN CARD FEATURES

The state machine below is a **candidate sketch, not an operative ceremony**. Atomic COMMIT, rollback-to-BLANK, key erasure, and tear-resistance are **claimed to WORK only after they are demonstrated on the exact card + applet build**. Read every "the card does X" as "a candidate design would do X (bench-required)".

```
S0 INIT      terminal opens session; card asserts state==BLANK
S1 ROLE      terminal writes role ∈ {B, C}; card records it; state=ROLED
S2 KEY       terminal writes the INDEPENDENT account xprv (m/48'/0'/0'/2' + chain code + origin fp),
              derived on the terminal from independent 256-bit entropy (NOT from Seed A or one master; the
              card RNG may contribute only a Multi-source sample). Card stores it, derives its account xpub,
              returns it; terminal compares to the pre-provision value. Applet does no BIP39/PBKDF2. state=KEYED
S3 A2        terminal writes the shared A2 (the SAME value onto both B and C); state=A2-SET
S4 SEAL      terminal sends the descriptor D (receive+change, 2-of-3 wsh(sortedmulti(2,A,B,C)), m/48'/0'/0'/2'/{0,1}/*);
              card verifies its own xpub ∈ descriptor, stores D + policy ID (D on BOTH B and C); state=SEALED
S5 COMMIT    single atomic write flips state=LIVE; card will now GET_A2 / SIGN / GET_POLICY
terminal side, after both B and C are LIVE and verified: WIPE A2 + all seed/root material from the terminal.
interruption at any Sx (power loss, yank): INTENT is resume from last committed state; a ceremony not
reaching S5 rolls back to BLANK – no partial-LIVE card, no duplicate seed, no export material left behind.
This resume/rollback is bench-required: not proven until demonstrated on the exact card + applet build.
```

EXPORT POLICY – SETTLED DIRECTION

The account **xprv (Signer B / Signer C)** is **never exportable**, by design (no export command, no export counter, no one-shot escape hatch). **A2 is exportable by design** — it is a shared bearer recovery key, not a Bitcoin signing key, and `GET_A2` releasing it is the normal recovery path. This is not tied to any timelock or spare-card fallback; those alternatives are rejected and appear only as a short appendix elsewhere in the dossier.

05 COMMAND SET

Minimal command set – six operations, one applet

ISO 7816-4 APDUs over T=1. Roles B and C run the **same minimal applet**; the role only changes which account key signs (D is stored on both). v1 adds no bus channel; where a future card supports a standard card-authenticated **SCP11** profile, operational commands could ride inside it as **later hardening only** (never pairing, never a recovery dependency). All multi-byte values big-endian.

SELECTED PRODUCT DIRECTION – NOT IMPLEMENTED; BENCH-GATED

This is the **settled** minimal surface, but every card-side behaviour is **bench-required (§02)**: capability, endurance, reset, tear, APDU size, and timing are Gate-B blockers. There is **no PIN, OAC, AUTHORIZE, lockout, RELEASE-SEED, Vault-Key, secure-channel requirement, or anti-exfil command** in the operative set. v1 adds no bus channel; a standard SCP11 profile is optional later hardening only. The retired custom channel and the deferred anti-exfil signing live in the non-normative appendices.

COMMAND	CLA INS	SM?	PURPOSE
---------	------------	-----	---------

SESSION BRING-UP			
SELECT	00 A4	no	select the QK2 applet by AID (same AID for B and C)
OPEN-SM (later hardening)	80 20	sets up	Not in v1 (bus is trusted). Optional later hardening only: where a future card supports a standard card-authenticated SCP11 profile, open it to protect the bus. No single-terminal pairing, no shared host secret, never a recovery dependency.
ATTEST (optional, Gate B)	80 30	opt	optional — signs a terminal challenge under a manufacturer card identity to help detect counterfeit cards. Manufacturing-security, not user authentication; never an online-revocation recovery dependency.
OPERATIONS			
GET_INFO	80 F0	opt	return role (B or C), protocol/applet version, lifecycle state, card id, wallet id, account fingerprint/origin, policy ID. Any counter reported is telemetry only — not clone-detection without an external anchor.
GET_A2	80 A2	opt	release the shared A2 ; possession of the card is the authorization. A2 is exportable by design — the terminal uses it to decrypt A1 (ChaCha20-Poly1305). Not a Bitcoin signing key.
SIGN	80 40	opt	bind wallet id + branch {0,1} + bounded child index + sighash32 + review_hash32 + session nonce, then sign with the non-exportable account key (Signer B/C): exact deterministic RFC6979 , strict DER, low-S, verify-before-release. Not a generic sign-any-32-bytes; the key never leaves the card.
GET_POLICY	80 50	opt	return the receive/change descriptor D , derivation profile, network, and wallet id. D is stored on both B and C in every tier (the redundant cloaked D kit copy is separate metadata, not the on-card D)
LIFECYCLE			
PROVISION	80 60	opt	atomic one-time load (BEGIN / DATA / COMMIT / ABORT) of role, the terminal-derived account xprv , A2, and D (the S0–S5 ceremony of §04); state==BLANK only
RETIRE-WIPE	80 6F	opt	best-effort field wipe of key material + A2 — NOT a reliable erasure guarantee on smart-card flash. Retirement is by on-chain sweep + physical destruction, not this command (bench-required)

No GET_A2 , SIGN , or GET_POLICY is gated behind a secret the card checks: **possession of the card is the authorization**, and the 2-of-3 threshold is the spending control. v1 adds no bus channel; a standard

CCP11 — Full Card Security — A Standard for Card-Based Cryptocurrency Transactions

SCP11 profile, if a future card ever supports one, is optional later hardening that only keeps the bus confidential and tamper-evident — never a recovery dependency.

06 THREAT → POSTURE

What the settled model does – and, deliberately, does not – try to stop

SELECTED POSTURE – NOT YET DEMONSTRATED

Each row is **SELECTED product direction, bench-required (§02)**. The card carries **no PIN, OAC, duress, lockout, or bricking**: a bearer card in hand is authorized, and the **2-of-3 threshold** is the spending control. Stealth is a first layer of defence, never a cryptographic factor. Do not read this table as a list of working protections.

ATTACK	SETTLED POSTURE (BENCH-REQUIRED)	WHERE
Bus tap reads a released A2 off the wire	in v1 the card bus is inside the trusted terminal boundary, so this is not a v1 attack surface; an optional later SCP11 profile would keep the SEC1210/bus ciphertext + MAC only (protects the bus, not the user)	§03
Shim injects / forges / replays a command	trusted-bus boundary in v1; T=1 framing must handle integrity/errors (not secret keys). Optional later SCP11 adds channel integrity + a sequence counter with out-of-order aborts	§03
Hostile / malicious terminal	Not solved. A single malicious terminal in an A1+B (or A1+C) session can obtain/control the normal threshold : it sees Seed A and A2 from A1 and can request B/C signatures. Cards give no independent display. The booted terminal + display path + firmware are one authorization trust domain ; 2-of-3 is storage/recovery resilience, not two-device authorization	§01
Cloned / counterfeit card	optional attestation only with a real manufacturing trust chain + rotation/revocation ; a monotonic counter is telemetry, not clone-detection without an external anchor + user action	§03

Stolen single card	one card is one signing factor; alone it cannot spend under 2-of-3. No PIN/OAC/lockout is offered or needed — that is the deliberate design for bearer objects (esp. inheritance card C)	\$01
Signer B / Signer C exfiltration off-card	the account key is never exportable ; SIGN verifies before release; exact deterministic RFC6979 + strict DER + low-S. Anti-exfil sign-to-contract is deferred, not a v1 blocker (Appendix B)	\$05
Power-cut mid-provisioning	transactional state machine; a ceremony not reaching COMMIT rolls back to BLANK — no partial-LIVE card, no duplicate seed (bench-required)	\$04
Loss of a card / the terminal	out of scope here — the 2-of-3 flows (A1+B, A1+C, B+C, replace terminal) live in the wallet spec; D survives on both cards and one redundant cloaked kit copy. Any lost/compromised factor means spend once with a surviving pair, then sweep into a fresh A'/B'/C'/A2'/D' kit — a factor is never reissued, since replacement changes the descriptor. Losing two signing factors can be unrecoverable.	—

Open decisions remaining

- **Card selection + capability matrix (\$02)** — the whole surface is gated on a real card that proves secp256k1, signing, AEAD/MAC, transactional writes, endurance, reset/tear, APDU size, and timing.
- **Whether to add any bus channel at all** — v1 treats the card bus as trusted and adds none; the open decision is whether a selected card exposes a usable standard card-authenticated SCP11 profile worth adding as later hardening (never pairing, never a recovery dependency). The custom channel (Appendix A) is **retired**, not a candidate.
- **Attestation trust chain** — only pursue attestation if a real manufacturing trust chain with rotation/revocation exists; otherwise treat identity as contingent. Attestation is manufacturing-security, not user-auth.
- **Anti-exfil (Appendix B)** — deferred; monitor whether the card ever exposes the nonce operations it needs. Not a v1 blocker.
- **SEC1210 bring-up (R2)** — prove ATR / APDU exchange / voltage class on the Microchip reference bench before PCB, as the hardware blueprint requires.

Custom secure channel – retired reference math

RETIRED – NOT A V1 MECHANISM AND NOT A CANDIDATE

v1 adds **no bespoke ECDH/AES/HMAC channel**; the card bus is inside the trusted terminal boundary (§01, §03). If bus hardening is ever pursued it would be a **standard card-authenticated SCP11 profile** (no terminal pairing, no shared host secret), **not** this custom construction. This appendix is retained only as a **retired reference fixture**; nothing here is selected or operative, it must never introduce single-terminal pairing, and a Python example proves math, not silicon.

COMPOSITION CAVEATS IF THIS FALLBACK IS EVER USED

AES-GCM is already an AEAD. The record must either use the **GCM tag alone** (retained and verified) or a reviewed **encrypt-then-MAC** — bolting a separate truncated HMAC onto GCM (as the values below do) is a non-standard composition needing cryptographic review. Derive **direction-separated** keys and nonce bases (terminal→card vs card→terminal), not the single shared set shown. Pin IV construction, sequence width/exhaustion, reset behaviour, max records, replay window, and the authenticated-header encoding byte-for-byte; **validate every received EC public key** (reject infinity / off-curve). State where the terminal static private key lives — on CM0 it has no hardware root a compromised application cannot impersonate — and note that binding a terminal static key must not become a recovery dependency or a pairing.

```
terminal T / card C – ephemeral E_T, E_C this insertion (reference only)
transcript  = SHA256( Q_T || Q_C || E_T || E_C )
              = 620b3e8f9641e65ddf024a28bb7df306197cfa660c8019c362209a716813cc8e
IKM         = x(e_T·E_C) || x(d_T·Q_C)      (card computes x(e_C·E_T) || x(d_C·Q_T) – equal)
ECDH shared = ee78462202e6d0e23eb06b2d77e564a87426acb7d3801bb4aa9c23b914df8ba9
K = HKDF-SHA256(salt=transcript, IKM, info="QK2-SC/v1", 76)
```

```
K = AES-256( K ) = c0cdd228bd82f566dab057ba72ccdf0b00b1ccf5fa12cc75c2a1fffb262f0dd2d0
```

```
K_enc (AES-256) = 00a0052000025000a00570e72e00f00000100151012a075e5e11f0202f900500
K_mac (HMAC)    = d82bf0bfe1b379ce2c0fd1d16197c0817a25ba843aa682610f6db3263e828769
IV base (GCM)   = b6b671cc582e44958779fdb0
-> both parties derive the same key (reference math only; NOT a card-feasibility proof)
```

WHY EPHEMERAL + ENDPOINT BINDING – WITHOUT PAIRING

Ephemeral-only ECDH stops a passive tap but not an active MITM on the bridge; binding endpoint keys into the shared secret authenticates the endpoints so a shim cannot interpose, and the ephemeral half gives forward secrecy. This endpoint binding is a **per-session bus protection**, not a durable card↔terminal pairing and not a spending factor.

APP. B DEFERRED

Anti-exfil leg-B signing – deferred, likely infeasible

DEFERRED – NOT A V1 BLOCKER

The account key (Signer B/C) signs on-card and is never exportable, but a malicious card could bias its ECDSA nonce to smuggle key bits into otherwise-valid signatures. Sign-to-contract is the candidate defense. It is **deferred**: the reference exchange below is math, very likely **not implementable** on a generic one-shot JavaCard ECDSA, and gated on the card exposing the required nonce operations (§02). Anti-exfil is explicitly **not a v1 blocker**.

FEASIBILITY BLOCKER

Generic one-shot JavaCard ECDSA cannot do this. Injecting a nonce tweak and returning the nonce point R_0 needs proprietary scalar/point operations the standard API does not expose; the referenced `secp256k1-zkp_ecdsa_s2c` module is itself experimental with an unstable API. If ever implemented, use an audited library, send the sighash in message 1, and treat repeated abnormal aborts as a reason to retire the session/card — success/failure, timing, and retry count can themselves leak.

```

1. terminal → card   host_commit = SHA256("QK2/s2c/host" || p)  AND the sighash   hides p
   host_commit = 62b4d0b2aa7f2638213ec4e87a0264b4c51d049069f3e35f8b10cded9261cedb
2. card → terminal   R0 = k0·G   where k0 = RFC6979(seedB, sighash || host_commit) [sighash from msg 1]
   R0 (opening) = 02536729d82e0da80b4b0a029aa642ceb0fb4e009dc300de00c57f41bc5b1ddb56
3. terminal → card   reveal p ; card checks host_commit, then signs with tweaked nonce
   p   = 11223344556677889900aabbccddeeff00112233445566778899aabbccdde01
   k   = k0 + SHA256("QK2/s2c/tweak" || R0 || p) mod n
   sig r = 2db45905b0f86fc787475b89523f12bc07aa3292652015210517d58ab6260c07
   sig s = 7679c589202ce05d65692be61206296fad73d9976ad18eb210fe6b6ef35a4318   (low-S)

terminal verify:  r == ( R0 + SHA256("...tweak..."||R0||p)·G ).x   AND   ECDSA(r,s) valid under Q_B
-> nonce-bound-to-terminal-entropy = TRUE   signature-valid = TRUE   (reference math; not on-card)

```

Even with this construction, **deterministic RFC6979 ECDSA does not make a leg exfiltration-proof** — malicious signer firmware can still encode data in a valid nonce. The real guarantee is structural: the **2-of-3 threshold means no single compromised signer alone can spend**. Anti-exfil, if it ever lands, only narrows a covert channel; it is not what secures the vault.

QK2 Card Protocol · working draft v1 · 2026-08-17

The non-normative appendix computations (the **retired** custom-channel ECDH key agreement and the **deferred** anti-exfil sign-to-contract) are **reference computations** from a pure-stdlib secp256k1 reference (cross-checked against BIP-32 TV1 and fingerprint `73c5da0a`); HKDF cross-checked against RFC 5869. These establish math only — they are **not a card-feasibility proof and do not close Gate B**, and neither is in the operative command set. Generator: `qk2_card_protocol.py`. Companion to the QuietKey QK2 Blueprint, QKEC-1 Specification, Standards Register, and PSBT Checklist.